

Real-Time Network Packet Analysis & Hybrid Network Intrusion Detection System (IDS)

Java-Based Network Threat Detection Using Pcap4J

Technical Project Report

Developed By

Alekhya Panguluri

Technologies Used

- **Java**
- **Pcap4J**
- **Npcap/libpcap**
- **Wireshark**
- **TCP/IP**
- **Nmap**
- **Hping3**
- **CSV Logging**

Executive Summary

Modern computer networks face a growing number of cyber threats, including reconnaissance scans, denial-of-service attacks, and unauthorized access attempts. Detecting these threats in real time is a fundamental requirement for protecting enterprise systems and maintaining network availability. Traditional Intrusion Detection Systems (IDS) often rely solely on signature-based detection, making them effective against known attacks but less capable of identifying previously unseen or evolving threats.

This project presents the design and implementation of a Hybrid Network Intrusion Detection System (IDS) developed in Java. The solution combines signature-based detection and anomaly-based detection to improve the identification of malicious network activity while reducing the limitations associated with using a single detection technique. Network packets are captured in real time using the Pcap4J library, analysed against predefined detection rules and behavioural thresholds, and recorded for security analysis.

To validate the effectiveness of the system, multiple controlled attack scenarios were performed using tools such as Nmap, hping3, and Wireshark. The IDS successfully detected reconnaissance scans, suspicious port activity, SYN flood attacks, and abnormal packet rates while generating detailed alerts stored in CSV format. Packet captures were also validated using Wireshark to ensure detection accuracy and confirm that generated alerts matched the observed network traffic.

This project demonstrates practical experience in network security, packet analysis, software development, and intrusion detection. It provides a strong foundation for understanding modern IDS design principles and highlights the ability to develop, test, and document a cybersecurity solution suitable for professional portfolio presentation.

Contents

Executive Summary.....	2
Project Overview.....	4
Problem Statement.....	5
Project Objectives & Technology Stack	6
Project Objectives	6
Technology Stack.....	6
System Architecture.....	7
Figure 1: System Architecture Diagram	8
Project Workflow.....	9
Implementation	9
Signature-Based Detection.....	10
Anomaly-Based Detection.....	11
Hybrid Detection Engine	11
Testing Methodology	12
Test Case 1: Nmap Port Scan Detection	13
Figure 2: Nmap Port Scan Against the Target System	13
Test Case 2: Real-Time IDS Monitoring	14
Figure 3: Real-Time IDS Console During Monitoring.....	14
Test Case 3: Alert Log Analysis.....	15
Figure 4: Generated Security Alerts Stored in CSV Format	15
Test Case 4: Suspicious Port Detection	15
Figure 5: Detection of Suspicious Port Connections	15
Test Case 5: Port Scan Detection	16
Figure 6: Behaviour-Based Port Scan Detection	16
Test Case 6: SYN Flood Detection	16
Figure 7: SYN Flood Attack Detection	16
Test Case 7: Packet Capture & Wireshark Validation.....	17
Figure 8: Captured Network Traffic	17
Figure 9: Wireshark Validation of Captured Packets	17
Results and Discussion	17
Challenges Faced	18

Skills Demonstrated	19
Future Improvements	20
Conclusion	21

Project Overview

Network Intrusion Detection Systems play an important role in identifying malicious activities before they develop into serious security incidents. These systems continuously monitor network traffic and analyse packets to determine whether communication patterns indicate suspicious or unauthorized behaviour. They are commonly deployed alongside firewalls and other defensive technologies to provide visibility into network activity and assist security analysts during investigations.

The objective of this project was to develop a lightweight Hybrid Network Intrusion Detection System capable of monitoring live network traffic and identifying common cyber-attacks in real time. Unlike traditional IDS solutions that depend exclusively on predefined attack signatures, this implementation combines signature-based techniques with anomaly-based analysis to improve detection capabilities across different attack scenarios.

The application was developed using Java and the Pcap4J packet capture library. During execution, network packets are captured directly from the selected network interface, analysed according to protocol-specific rules, and evaluated against behavioural thresholds designed to identify suspicious activity. Security events are recorded in CSV format, allowing analysts to review generated alerts after testing or during incident investigations.

To evaluate the effectiveness of the system, controlled attack simulations were performed using recognised cybersecurity tools including Nmap, hping3, and Wireshark. These experiments demonstrate how the IDS responds to reconnaissance scanning, suspicious port connections, SYN flood attacks, and other network anomalies while validating captured traffic against packet-level analysis.

Problem Statement

Cyber-attacks continue to increase in both frequency and complexity, making continuous network monitoring an essential component of modern cybersecurity. While firewalls help prevent unauthorised access, they cannot detect every type of malicious activity occurring within a network. Attackers frequently perform reconnaissance scans, exploit vulnerable services, or generate abnormal traffic patterns that require additional monitoring beyond traditional perimeter security.

Many intrusion detection systems rely exclusively on signature-based detection, which compares network activity against a database of known attack patterns. Although this approach provides reliable detection for recognised threats, it cannot easily identify new or modified attack techniques that do not match existing signatures. Conversely, purely anomaly-based systems may detect unknown threats but often generate a higher number of false positives because normal network behaviour varies between environments.

The goal of this project was to address these limitations by implementing a **hybrid detection model** that combines both signature-based and anomaly-based analysis. By integrating these two approaches, the IDS can accurately detect known attacks while also recognising unusual network behaviour that may indicate previously unseen threats or ongoing reconnaissance activities.

Project Objectives & Technology Stack

Project Objectives

The primary objectives of this project were to:

- Develop a Java-based Hybrid Intrusion Detection System capable of analysing live network traffic.
- Capture packets in real time using the Pcap4J packet capture framework.
- Detect common cyber-attacks using signature-based detection rules.
- Identify abnormal network behaviour using anomaly-based detection techniques.
- Generate detailed security alerts for suspicious activities.
- Store alerts in CSV format for future investigation and reporting.
- Validate packet captures using Wireshark.
- Demonstrate the effectiveness of the IDS using controlled attack simulations.

Technology Stack

Technology	Purpose
Java	Core application development
Pcap4J	Real-time packet capture
Npcap / libpcap	Packet capture driver
Wireshark	Packet validation and traffic analysis
Nmap	Port scanning and reconnaissance testing
hping3	SYN flood and packet generation
CSV Logging	Security alert recording
TCP/IP Protocol Suite	Network communication analysis

System Architecture

The Hybrid Network Intrusion Detection System (IDS) was designed using a modular architecture to ensure that packet capture, traffic analysis, detection logic, and alert generation operate independently while working together as a complete monitoring solution. This modular approach improves scalability, maintainability, and makes it easier to extend the system with additional detection techniques in the future.

At the core of the system is the Packet Capture Module, which continuously monitors a selected network interface using the Pcap4J library. Every captured packet is decoded to extract relevant protocol information such as source IP address, destination IP address, protocol type, port numbers, TCP flags, and timestamps. This information forms the basis for subsequent security analysis.

Once packet information has been extracted, it is processed simultaneously by two independent detection engines. The Signature-Based Detection Engine compares network traffic against predefined attack signatures such as TCP NULL scans, TCP XMAS scans, and suspicious port connections. In parallel, the Anomaly-Based Detection Engine analyses traffic behaviour over time to identify unusual communication patterns including excessive packet rates, port scanning behaviour, and SYN flood attacks. Combining these two approaches increases overall detection capability while reducing the limitations associated with relying on a single detection method.

Whenever suspicious activity is detected, the Alert Generation Module records detailed information including timestamps, attack type, protocol, source address, destination address, severity level, and descriptive alert messages. These alerts are stored in CSV format for future investigation and can be correlated with packet captures in Wireshark to validate the accuracy of the detection process.

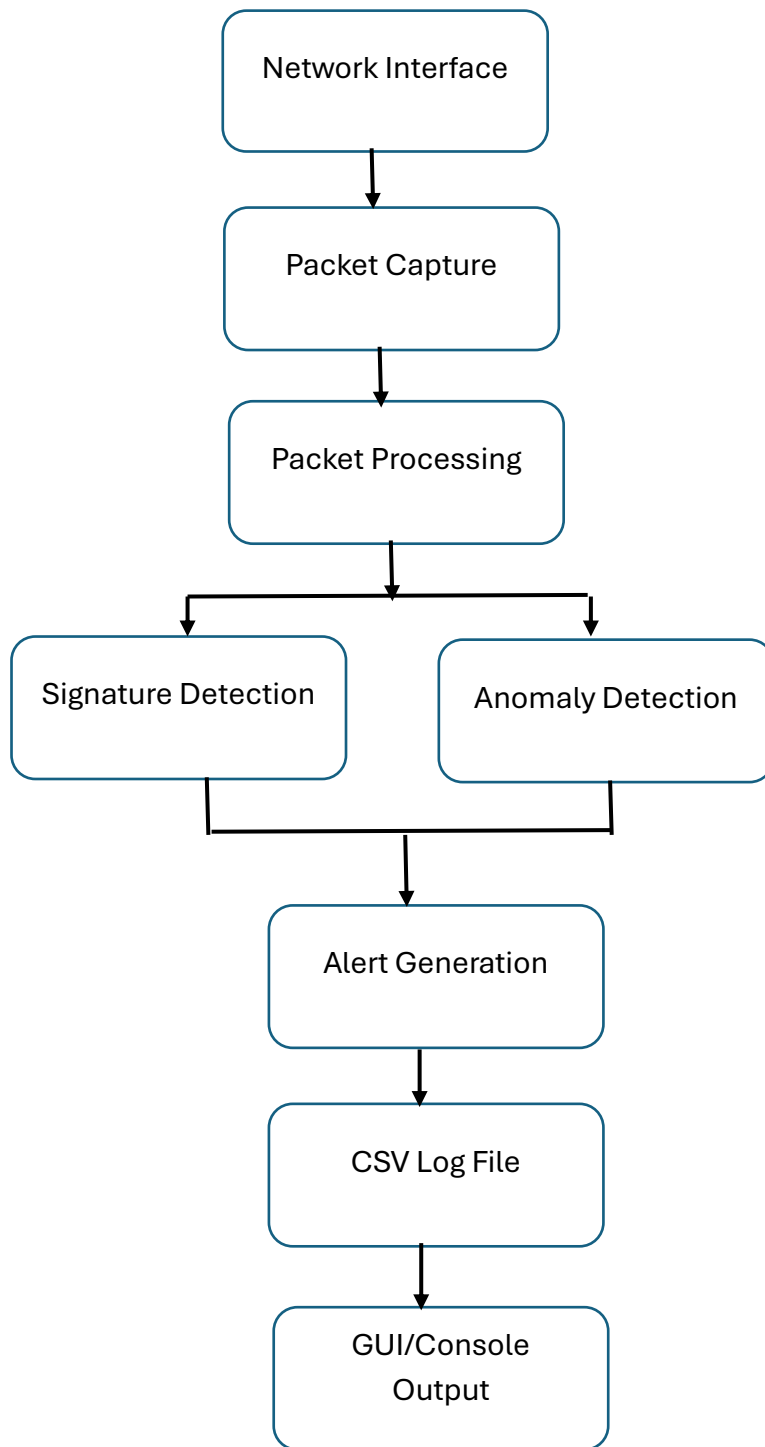


Figure 1: System Architecture Diagram

The architecture diagram illustrates the complete workflow of the Hybrid IDS from packet acquisition through to alert generation. It highlights the separation between packet capture, traffic analysis, detection logic, and logging components, making the design easy to understand and maintain. This modular architecture also allows additional detection algorithms to be integrated without significantly modifying the existing implementation.

Project Workflow

The operation of the Hybrid Intrusion Detection System follows a structured workflow that enables continuous monitoring of network activity while ensuring that every packet is analysed before alerts are generated. Each processing stage contributes to identifying suspicious behaviour and recording security events in real time.

The workflow begins with the initialization of the packet capture interface. Using Pcap4J, the application connects to the selected network adapter and continuously listens for incoming and outgoing packets. As packets are received, protocol headers are parsed to extract network attributes including IP addresses, port numbers, TCP flags, packet size, and protocol information.

The extracted packet information is forwarded to the detection engine where signature-based and anomaly-based analysis occur simultaneously. Signature matching identifies known attack patterns while behavioural analysis evaluates traffic characteristics over configurable time windows to detect abnormal network activity.

When malicious activity is confirmed, an alert is immediately generated and written to the alert log. Security analysts can then review these alerts alongside captured packets using Wireshark, providing additional evidence for investigation and validation.

Workflow Steps

1. Start IDS Application
2. Select Network Interface
3. Capture Live Packets
4. Extract Packet Information
5. Signature Analysis
6. Behaviour Analysis
7. Generate Alert
8. Store Alert in CSV
9. Validate Using Wireshark
10. Continue Monitoring

Implementation

The Hybrid Network Intrusion Detection System was implemented using Java, providing a platform-independent and object-oriented environment for network traffic analysis. The project utilises the Pcap4J library to capture packets directly from the selected network interface while allowing detailed inspection of protocol headers.

The implementation follows a modular programming approach where individual components perform specific responsibilities. Packet capture operates independently

from detection logic, enabling the IDS to process incoming traffic continuously without interrupting analysis. This modular structure also simplifies future enhancements such as additional attack signatures or machine learning integration.

Captured packets are decoded to retrieve protocol-specific information including Ethernet headers, IP addresses, transport protocols, TCP flags, source ports, destination ports, packet lengths, and timestamps. These attributes are then evaluated against predefined security rules to determine whether malicious behaviour is present.

Alert information is recorded in CSV format, providing a structured log that can be reviewed using spreadsheet software or imported into security monitoring platforms. The implementation also allows packet captures to be validated using Wireshark, ensuring that IDS alerts correspond accurately to observed network traffic.

Key Implementation Features

- Real-time packet capture
- Multi-threaded packet processing
- Signature matching engine
- Behaviour analysis engine
- CSV alert logging
- Packet validation
- Modular design

Signature-Based Detection

Signature-based detection identifies malicious traffic by comparing captured packets against predefined attack patterns. This method provides reliable detection for well-known attacks that exhibit characteristic packet structures or protocol behaviour.

Within this project, several commonly encountered reconnaissance techniques were implemented as detection signatures. TCP NULL scans are identified by detecting packets where no TCP control flags are set. TCP XMAS scans are recognised through the simultaneous activation of FIN, PSH, and URG flags. Suspicious connections targeting commonly abused ports are also detected through predefined monitoring rules.

The advantage of signature-based detection is its high accuracy for recognised attacks and relatively low false-positive rate. However, it cannot identify previously unseen attacks that do not match stored signatures. For this reason, the Hybrid IDS combines signature analysis with behavioural anomaly detection to improve overall detection performance.

Implemented Signatures

- TCP NULL Scan
- TCP XMAS Scan
- Suspicious Port Access
- Abnormal TCP Flag Combinations

Anomaly-Based Detection

Unlike signature-based techniques, anomaly-based detection focuses on identifying unusual network behaviour rather than matching known attack patterns. The IDS establishes behavioural thresholds and monitors traffic over time to determine whether communication patterns deviate from expected activity.

During implementation, several behavioural metrics were monitored including packet rate, connection frequency, SYN packet counts, and the number of destination ports contacted by individual hosts. When predefined thresholds are exceeded, the IDS generates an anomaly alert indicating possible malicious activity.

This approach enables the detection of previously unknown attacks while providing additional protection against reconnaissance and denial-of-service techniques that may not exactly match predefined signatures.

Behavioural Indicators

- High Packet Rate
- SYN Flood Activity
- Port Scanning Behaviour
- Repeated Connection Attempts
- Excessive Destination Ports

Hybrid Detection Engine

The primary strength of this project lies in combining Signature-Based Detection and Anomaly-Based Detection within a single monitoring platform. Each technique compensates for the limitations of the other, resulting in improved overall detection capability.

Signature analysis provides rapid and highly accurate identification of recognised attacks, while anomaly detection identifies suspicious behavioural changes that may indicate new or evolving threats. The integration of these techniques creates a balanced

detection strategy capable of monitoring both known attack signatures and abnormal traffic behaviour simultaneously.

This hybrid approach significantly improves the effectiveness of the IDS and demonstrates a practical understanding of modern intrusion detection principles used within enterprise security environments.

Testing Methodology

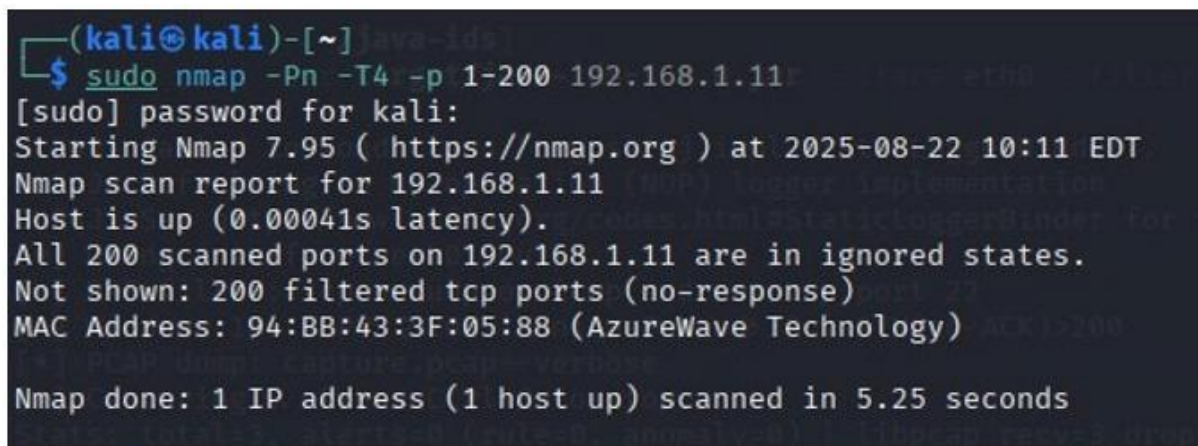
To evaluate the effectiveness of the Hybrid Network Intrusion Detection System (IDS), a series of controlled attack simulations were performed in a dedicated testing environment. The objective was to verify that the IDS could accurately detect common network attacks while maintaining continuous packet monitoring and generating detailed security alerts.

The testing environment consisted of two virtual machines connected within the same local network. One system acted as the attacker, generating malicious traffic using penetration testing tools, while the second system hosted the Hybrid IDS and monitored incoming network packets. This isolated environment allowed multiple attack scenarios to be executed safely without affecting external systems.

Several widely used cybersecurity tools were employed during testing. Nmap was used to perform reconnaissance and port scanning activities, hping3 generated SYN flood traffic, and Wireshark captured network packets for validation. Each detected event was logged by the IDS into a CSV file, enabling comparison between captured traffic and generated alerts.

The testing process focused on validating both signature-based detection and anomaly-based detection. Known attack patterns were identified through predefined signatures, while abnormal network behaviour such as excessive packet rates and repeated connection attempts was detected using behavioural analysis. The combination of these techniques provided comprehensive visibility into network activity and demonstrated the effectiveness of the hybrid detection approach.

Test Case 1: Nmap Port Scan Detection

A terminal window with a dark background and light-colored text. The prompt is (kali@kali)-[~]. The user enters the command \$ sudo nmap -Pn -T4 -p 1-200 192.168.1.11. The system prompts for a password, which is entered. The output shows Nmap 7.95 starting at 2025-08-22 10:11 EDT. The scan report for 192.168.1.11 indicates the host is up with a latency of 0.00041s. All 200 scanned ports are in ignored states. Not shown: 200 filtered tcp ports (no-response). The MAC address is 94:BB:43:3F:05:88 (AzureWave Technology). The scan is done in 5.25 seconds.

```
(kali@kali)-[~]
$ sudo nmap -Pn -T4 -p 1-200 192.168.1.11
[sudo] password for kali:
Starting Nmap 7.95 ( https://nmap.org ) at 2025-08-22 10:11 EDT
Nmap scan report for 192.168.1.11
Host is up (0.00041s latency).
All 200 scanned ports on 192.168.1.11 are in ignored states.
Not shown: 200 filtered tcp ports (no-response)
MAC Address: 94:BB:43:3F:05:88 (AzureWave Technology)

Nmap done: 1 IP address (1 host up) scanned in 5.25 seconds
```

Figure 2: Nmap Port Scan Against the Target System

The screenshot above shows the attacker system executing an Nmap TCP port scan against the target machine. Nmap is one of the most commonly used reconnaissance tools in penetration testing and is frequently employed by attackers during the initial stages of an intrusion. By probing multiple ports, an attacker can identify running services, open ports, and potential vulnerabilities before attempting exploitation.

During this test, the Hybrid IDS continuously monitored network traffic generated by the scan. As packets arrived, the packet capture module extracted protocol information including source IP addresses, destination IP addresses, TCP flags, destination ports, and timestamps. These packet attributes were forwarded to the detection engine for further analysis.

The anomaly detection engine recognised that a single source host was attempting connections to a large number of destination ports within a short period. Since this behaviour exceeded the configured detection threshold, the IDS generated a Port Scan Alert. This confirms that the system successfully identifies reconnaissance activities that commonly precede cyber-attacks.

Test Case 2: Real-Time IDS Monitoring

```
ls
bradnewfile.txt Desktop Documents Downloads hello.txt hey.txt java-ids Music newfile.txt Pictures Public python real_time_ids Templates Videos

(kali@kali)~$ cd java-ids
(kali@kali)~/java-ids$ sudo java -jar target/java-ids-1.0.0.jar -iface eth0 -filter 'tcp or udp or icmp and not tcp port 22' -pcap capture.pcap--verbose
[sudo] password for kali:
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
[*] Using interface: eth0 (null)
[*] BPF filter: tcp or udp or icmp and not tcp port 22
[*] Window: 18s | PPS ≥ 200 | UniquePorts ≥ 30 | SYN(no-ACK)>200
[*] PCAP dump: capture.pcap--verbose
[*] Capturing... Press Ctrl+C to stop.
Stats: total=3, alerts=0 (rule=0, anomaly=0) | libpcap rcv=3 dropped=3 ifdrop=0
Stats: total=4, alerts=0 (rule=0, anomaly=0) | libpcap rcv=4 dropped=4 ifdrop=0
Stats: total=4, alerts=0 (rule=0, anomaly=0) | libpcap rcv=4 dropped=4 ifdrop=0
Stats: total=4, alerts=0 (rule=0, anomaly=0) | libpcap rcv=4 dropped=4 ifdrop=0
Stats: total=5, alerts=0 (rule=0, anomaly=0) | libpcap rcv=5 dropped=5 ifdrop=0
Stats: total=9, alerts=0 (rule=0, anomaly=0) | libpcap rcv=9 dropped=9 ifdrop=0
Stats: total=9, alerts=0 (rule=0, anomaly=0) | libpcap rcv=9 dropped=9 ifdrop=0
Stats: total=9, alerts=0 (rule=0, anomaly=0) | libpcap rcv=9 dropped=9 ifdrop=0
2025-08-22T14:11:51.051378932Z [SIGNATURE,SUSPICIOUS_PORT/LOW] 192.168.1.22:50511 → 192.168.1.11:23 TCP | Connection to common risky port 23
2025-08-22T14:11:51.060450148Z [SIGNATURE,SUSPICIOUS_PORT/LOW] 192.168.1.22:50511 → 192.168.1.11:19 TCP | Connection to common risky port 19
2025-08-22T14:11:51.759477161Z [SIGNATURE,SUSPICIOUS_PORT/LOW] 192.168.1.22:50511 → 192.168.1.11:23 TCP | Connection to common risky port 23
2025-08-22T14:11:51.760714373Z [SIGNATURE,SUSPICIOUS_PORT/LOW] 192.168.1.22:50511 → 192.168.1.11:19 TCP | Connection to common risky port 19
2025-08-22T14:11:52.055411203Z [ANOMALY,PORT_SCAN/HIGH] 192.168.1.22:50511 → 192.168.1.11:195 TCP | Multiple destination ports from source (30 unique in 10s)
2025-08-22T14:11:52.408331442Z [ANOMALY,PORT_SCAN/HIGH] 192.168.1.22:50511 → 192.168.1.11:128 TCP | Multiple destination ports from source (30 unique in 10s)
2025-08-22T14:11:52.800854627Z [ANOMALY,PORT_SCAN/HIGH] 192.168.1.22:50511 → 192.168.1.11:43 TCP | Multiple destination ports from source (30 unique in 10s)
2025-08-22T14:11:53.279396045Z [ANOMALY,PORT_SCAN/HIGH] 192.168.1.22:50511 → 192.168.1.11:51 TCP | Multiple destination ports from source (30 unique in 10s)
2025-08-22T14:11:53.608845051Z [ANOMALY,HIGH_PPS/MEDIUM] 192.168.1.22:50511 → 192.168.1.11:11 TCP | High packet rate from source (200 pkt/10s)
2025-08-22T14:11:53.608845051Z [ANOMALY,SYN_FLOOD_WINT/CRITICAL] 192.168.1.22:50511 → 192.168.1.11:11 TCP | Excessive SYN (no-ACK) from source (200 in 10s)
2025-08-22T14:11:53.676258760Z [ANOMALY,PORT_SCAN/HIGH] 192.168.1.22:50511 → 192.168.1.11:89 TCP | Multiple destination ports from source (30 unique in 10s)
2025-08-22T14:11:54.087787052Z [ANOMALY,PORT_SCAN/HIGH] 192.168.1.22:50511 → 192.168.1.11:27 TCP | Multiple destination ports from source (30 unique in 10s)
2025-08-22T14:11:54.487540532Z [ANOMALY,PORT_SCAN/HIGH] 192.168.1.22:50511 → 192.168.1.11:167 TCP | Multiple destination ports from source (30 unique in 10s)
Stats: total=348, alerts=12 (rule=4, anomaly=8) | libpcap rcv=336 dropped=353 ifdrop=0
2025-08-22T14:11:54.892455718Z [ANOMALY,PORT_SCAN/HIGH] 192.168.1.22:50511 → 192.168.1.11:182 TCP | Multiple destination ports from source (30 unique in 10s)
2025-08-22T14:11:55.284555233Z [ANOMALY,PORT_SCAN/HIGH] 192.168.1.22:50511 → 192.168.1.11:45 TCP | Multiple destination ports from source (30 unique in 10s)
Stats: total=421, alerts=14 (rule=4, anomaly=10) | libpcap rcv=407 dropped=421 ifdrop=0
Stats: total=421, alerts=14 (rule=4, anomaly=10) | libpcap rcv=417 dropped=421 ifdrop=0
Stats: total=431, alerts=14 (rule=4, anomaly=10) | libpcap rcv=417 dropped=431 ifdrop=0
Stats: total=432, alerts=14 (rule=4, anomaly=10) | libpcap rcv=418 dropped=432 ifdrop=0
Stats: total=433, alerts=14 (rule=4, anomaly=10) | libpcap rcv=419 dropped=433 ifdrop=0
Stats: total=433, alerts=14 (rule=4, anomaly=10) | libpcap rcv=419 dropped=433 ifdrop=0
Stats: total=433, alerts=14 (rule=4, anomaly=10) | libpcap rcv=419 dropped=433 ifdrop=0
Stats: total=434, alerts=14 (rule=4, anomaly=10) | libpcap rcv=419 dropped=434 ifdrop=0
Stats: total=435, alerts=14 (rule=4, anomaly=10) | libpcap rcv=421 dropped=435 ifdrop=0
Stats: total=435, alerts=14 (rule=4, anomaly=10) | libpcap rcv=431 dropped=435 ifdrop=0
```

Figure 3: Real-Time IDS Console During Monitoring

This screenshot displays the Hybrid IDS actively monitoring live network traffic while processing captured packets in real time. The console output provides continuous updates regarding packet capture statistics, generated alerts, detection rules, and runtime information.

As attack traffic was introduced into the network, the IDS immediately identified multiple suspicious activities including Suspicious Port Connections, Port Scan Detection, High Packet Rate Alerts, and SYN Flood Indicators. Each event was categorised according to its severity level and displayed in the console with timestamps and network information.

The successful generation of multiple alerts demonstrates that the system is capable of processing high volumes of network traffic while maintaining real-time visibility into security events. This validates both the packet capture mechanism and the hybrid detection engine.

Test Case 3: Alert Log Analysis

```
(kali@kali)-[~]
└─$ ls
brandnewfile.txt  Desktop  Documents  Downloads  hello.txt  hey.txt  java-ids  Music  newfile.txt  Pictures  Public  python  real_time_ids  Templates  Videos

(kali@kali)-[~]
└─$ cd java-ids

(kali@kali)-[~/java-ids]
└─$ ls
alerts.csv  capture.pcap  capture.pcap--verbose  pom.xml  src  target  traffic.csv

(kali@kali)-[~/java-ids]
└─$ tail -f alerts.csv
2025-08-22T14:11:54.891578604Z,ANOMALY.PORT_SCAN,HIGH,192.168.1.22,56511,192.168.1.11,182,TCP,58,"Multiple destination ports from source (30 unique in 10s)"
2025-08-22T14:11:54.891778633Z,ANOMALY.PORT_SCAN,HIGH,192.168.1.22,56511,192.168.1.11,182,TCP,58,"Multiple destination ports from source (30 unique in 10s)"
2025-08-22T14:11:54.892455718Z,ANOMALY.PORT_SCAN,HIGH,192.168.1.22,56511,192.168.1.11,182,TCP,58,"Multiple destination ports from source (30 unique in 10s)"
2025-08-22T14:11:55.284031167Z,ANOMALY.PORT_SCAN,HIGH,192.168.1.22,56511,192.168.1.11,45,TCP,58,"Multiple destination ports from source (30 unique in 10s)"
2025-08-22T14:11:55.284062431Z,ANOMALY.PORT_SCAN,HIGH,192.168.1.22,56511,192.168.1.11,45,TCP,58,"Multiple destination ports from source (30 unique in 10s)"
2025-08-22T14:11:55.284286356Z,ANOMALY.PORT_SCAN,HIGH,192.168.1.22,56511,192.168.1.11,45,TCP,58,"Multiple destination ports from source (30 unique in 10s)"
2025-08-22T14:11:55.283951560Z,ANOMALY.PORT_SCAN,HIGH,192.168.1.22,56511,192.168.1.11,45,TCP,58,"Multiple destination ports from source (30 unique in 10s)"
2025-08-22T14:11:55.284555233Z,ANOMALY.PORT_SCAN,HIGH,192.168.1.22,56511,192.168.1.11,45,TCP,58,"Multiple destination ports from source (30 unique in 10s)"
2025-08-22T14:11:55.285620678Z,ANOMALY.PORT_SCAN,HIGH,192.168.1.22,56511,192.168.1.11,45,TCP,58,"Multiple destination ports from source (30 unique in 10s)"
```

Figure 4: Generated Security Alerts Stored in CSV Format

Following the detection of malicious activity, the IDS automatically records all alerts in a structured CSV file. Each entry contains valuable forensic information including timestamps, alert type, severity level, source IP address, destination IP address, destination port, protocol, and a detailed description of the detected event.

Maintaining a structured alert log provides security analysts with a permanent record of detected threats, enabling incident investigation, historical analysis, and security reporting. CSV logging also allows alerts to be imported into SIEM platforms or spreadsheet tools for further analysis.

The screenshot confirms that the IDS successfully recorded multiple security events while maintaining consistent formatting and detailed event descriptions. This demonstrates the effectiveness of the logging component within the overall system architecture.

Test Case 4: Suspicious Port Detection

```
2025-08-22T10:18:33.029044505Z,SIGNATURE.SUSPICIOUS_PORT,LOW,192.168.1.22,52587,192.168.1.11,23,TCP,58,"Connection to common risky port 23"
2025-08-22T10:18:33.042790394Z,SIGNATURE.SUSPICIOUS_PORT,LOW,192.168.1.22,52587,192.168.1.11,23,TCP,58,"Connection to common risky port 23"
2025-08-22T10:18:33.041395918Z,SIGNATURE.SUSPICIOUS_PORT,LOW,192.168.1.22,52587,192.168.1.11,23,TCP,58,"Connection to common risky port 23"
2025-08-22T10:18:33.045909878Z,SIGNATURE.SUSPICIOUS_PORT,LOW,192.168.1.22,52587,192.168.1.11,23,TCP,58,"Connection to common risky port 23"
2025-08-22T10:18:33.051603197Z,SIGNATURE.SUSPICIOUS_PORT,LOW,192.168.1.22,52587,192.168.1.11,23,TCP,58,"Connection to common risky port 23"
2025-08-22T10:18:33.051637180Z,SIGNATURE.SUSPICIOUS_PORT,LOW,192.168.1.22,52587,192.168.1.11,23,TCP,58,"Connection to common risky port 23"
2025-08-22T10:18:34.123511827Z,SIGNATURE.SUSPICIOUS_PORT,LOW,192.168.1.22,52589,192.168.1.11,23,TCP,58,"Connection to common risky port 23"
2025-08-22T10:18:34.127722360Z,SIGNATURE.SUSPICIOUS_PORT,LOW,192.168.1.22,52589,192.168.1.11,23,TCP,58,"Connection to common risky port 23"
2025-08-22T10:18:34.130865009Z,SIGNATURE.SUSPICIOUS_PORT,LOW,192.168.1.22,52589,192.168.1.11,23,TCP,58,"Connection to common risky port 23"
2025-08-22T10:18:34.134671091Z,SIGNATURE.SUSPICIOUS_PORT,LOW,192.168.1.22,52589,192.168.1.11,23,TCP,58,"Connection to common risky port 23"
2025-08-22T10:18:34.135172384Z,SIGNATURE.SUSPICIOUS_PORT,LOW,192.168.1.22,52589,192.168.1.11,23,TCP,58,"Connection to common risky port 23"
2025-08-22T10:18:34.135144756Z,SIGNATURE.SUSPICIOUS_PORT,LOW,192.168.1.22,52589,192.168.1.11,23,TCP,58,"Connection to common risky port 23"
```

Figure 5: Detection of Suspicious Port Connections

Certain TCP ports are frequently targeted during malicious activities due to the services they host. The Hybrid IDS continuously monitors traffic destined for these commonly abused ports and generates alerts whenever suspicious connections are detected.

During testing, network traffic targeting monitored ports was generated intentionally. The Signature-Based Detection Engine successfully matched these packets against

predefined detection rules and generated immediate alerts indicating suspicious port activity.

This test demonstrates the effectiveness of signature-based detection in recognising known attack patterns with high accuracy while maintaining a low false-positive rate.

Test Case 5: Port Scan Detection

```
2025-08-22T10:18:36.278124891Z,ANOMALY.PORT_SCAN,HIGH,192.168.1.22,52587,192.168.1.11,137,TCP,58,"Multiple destination ports from source (30 unique in 10s)"
2025-08-22T10:18:36.278409814Z,ANOMALY.PORT_SCAN,HIGH,192.168.1.22,52587,192.168.1.11,137,TCP,58,"Multiple destination ports from source (30 unique in 10s)"
2025-08-22T10:18:36.282154929Z,ANOMALY.PORT_SCAN,HIGH,192.168.1.22,52587,192.168.1.11,137,TCP,58,"Multiple destination ports from source (30 unique in 10s)"
2025-08-22T10:18:36.282211446Z,ANOMALY.PORT_SCAN,HIGH,192.168.1.22,52587,192.168.1.11,137,TCP,58,"Multiple destination ports from source (30 unique in 10s)"
2025-08-22T10:18:36.282355370Z,ANOMALY.PORT_SCAN,HIGH,192.168.1.22,52587,192.168.1.11,137,TCP,58,"Multiple destination ports from source (30 unique in 10s)"
2025-08-22T10:18:36.678094204Z,ANOMALY.PORT_SCAN,HIGH,192.168.1.22,52587,192.168.1.11,181,TCP,58,"Multiple destination ports from source (30 unique in 10s)"
2025-08-22T10:18:36.679607687Z,ANOMALY.PORT_SCAN,HIGH,192.168.1.22,52587,192.168.1.11,181,TCP,58,"Multiple destination ports from source (30 unique in 10s)"
2025-08-22T10:18:36.688915587Z,ANOMALY.PORT_SCAN,HIGH,192.168.1.22,52587,192.168.1.11,181,TCP,58,"Multiple destination ports from source (30 unique in 10s)"
2025-08-22T10:18:36.700725391Z,ANOMALY.PORT_SCAN,HIGH,192.168.1.22,52587,192.168.1.11,181,TCP,58,"Multiple destination ports from source (30 unique in 10s)"
2025-08-22T10:18:36.792806351Z,ANOMALY.PORT_SCAN,HIGH,192.168.1.22,52587,192.168.1.11,181,TCP,58,"Multiple destination ports from source (30 unique in 10s)"
2025-08-22T10:18:37.094331238Z,ANOMALY.PORT_SCAN,HIGH,192.168.1.22,52587,192.168.1.11,163,TCP,58,"Multiple destination ports from source (30 unique in 10s)"
2025-08-22T10:18:37.098027827Z,ANOMALY.PORT_SCAN,HIGH,192.168.1.22,52587,192.168.1.11,163,TCP,58,"Multiple destination ports from source (30 unique in 10s)"
2025-08-22T10:18:37.102636353Z,ANOMALY.PORT_SCAN,HIGH,192.168.1.22,52587,192.168.1.11,163,TCP,58,"Multiple destination ports from source (30 unique in 10s)"
2025-08-22T10:18:37.106776668Z,ANOMALY.PORT_SCAN,HIGH,192.168.1.22,52587,192.168.1.11,163,TCP,58,"Multiple destination ports from source (30 unique in 10s)"
```

Figure 6: Behaviour-Based Port Scan Detection

Port scanning represents one of the earliest stages of network reconnaissance. Rather than relying solely on packet signatures, the Hybrid IDS analyses connection behaviour over configurable time windows.

The anomaly detection engine tracks the number of unique destination ports contacted by individual source hosts. When this value exceeds the configured threshold, the IDS generates a **Port Scan Alert** indicating abnormal reconnaissance activity.

The screenshot confirms that the IDS successfully identified multiple destination port connections from a single source within a short period, validating the effectiveness of the behavioural detection algorithm.

Test Case 6: SYN Flood Detection

```
2025-08-22T10:18:36.042006607Z,ANOMALY.SYN_FLOOD,HIGH,CRITICAL,192.168.1.22,52587,192.168.1.11,119,TCP,58,"Excessive SYN (no-ACK) from source (200 in 10s)"
2025-08-22T10:18:36.042044571Z,ANOMALY.SYN_FLOOD,HIGH,CRITICAL,192.168.1.22,52587,192.168.1.11,119,TCP,58,"Excessive SYN (no-ACK) from source (200 in 10s)"
2025-08-22T10:18:36.042082435Z,ANOMALY.SYN_FLOOD,HIGH,CRITICAL,192.168.1.22,52587,192.168.1.11,119,TCP,58,"Excessive SYN (no-ACK) from source (200 in 10s)"
2025-08-22T10:18:36.042120299Z,ANOMALY.SYN_FLOOD,HIGH,CRITICAL,192.168.1.22,52587,192.168.1.11,119,TCP,58,"Excessive SYN (no-ACK) from source (200 in 10s)"
2025-08-22T10:18:36.042158163Z,ANOMALY.SYN_FLOOD,HIGH,CRITICAL,192.168.1.22,52587,192.168.1.11,119,TCP,58,"Excessive SYN (no-ACK) from source (200 in 10s)"
2025-08-22T10:18:36.042196027Z,ANOMALY.SYN_FLOOD,HIGH,CRITICAL,192.168.1.22,52587,192.168.1.11,119,TCP,58,"Excessive SYN (no-ACK) from source (200 in 10s)"
2025-08-22T10:18:36.042233891Z,ANOMALY.SYN_FLOOD,HIGH,CRITICAL,192.168.1.22,52587,192.168.1.11,119,TCP,58,"Excessive SYN (no-ACK) from source (200 in 10s)"
2025-08-22T10:18:36.042271755Z,ANOMALY.SYN_FLOOD,HIGH,CRITICAL,192.168.1.22,52587,192.168.1.11,119,TCP,58,"Excessive SYN (no-ACK) from source (200 in 10s)"
2025-08-22T10:18:36.042309619Z,ANOMALY.SYN_FLOOD,HIGH,CRITICAL,192.168.1.22,52587,192.168.1.11,119,TCP,58,"Excessive SYN (no-ACK) from source (200 in 10s)"
2025-08-22T10:18:36.042347483Z,ANOMALY.SYN_FLOOD,HIGH,CRITICAL,192.168.1.22,52587,192.168.1.11,119,TCP,58,"Excessive SYN (no-ACK) from source (200 in 10s)"
```

Figure 7: SYN Flood Attack Detection

A SYN Flood attack attempts to exhaust server resources by transmitting large numbers of TCP SYN packets without completing the three-way handshake. This technique is commonly used during Denial-of-Service (DoS) attacks.

The Hybrid IDS continuously monitors incoming SYN packets and maintains counters for each source host. When the observed packet rate exceeds predefined thresholds, the anomaly detection engine generates a **Critical SYN Flood Alert**.

The generated alert confirms that the system successfully recognised excessive SYN traffic and accurately classified the event as a high-severity security incident.

Test Case 7: Packet Capture & Wireshark Validation

Time	Source	Destination	Protocol	Flags
2025-08-22T10:18:36.778821283Z	192.168.1.22	192.168.1.11	TCP	SYN
2025-08-22T10:18:36.779031908Z	192.168.1.22	192.168.1.11	TCP	SYN
2025-08-22T10:18:36.882123727Z	192.168.1.22	192.168.1.11	TCP	SYN
2025-08-22T10:18:36.882677861Z	192.168.1.22	192.168.1.11	TCP	SYN
2025-08-22T10:18:36.8829975165Z	192.168.1.22	192.168.1.11	TCP	SYN
2025-08-22T10:18:36.883221656Z	192.168.1.22	192.168.1.11	TCP	SYN
2025-08-22T10:18:36.883489559Z	192.168.1.22	192.168.1.11	TCP	SYN
2025-08-22T10:18:36.883823791Z	192.168.1.22	192.168.1.11	TCP	SYN
2025-08-22T10:18:36.884102096Z	192.168.1.22	192.168.1.11	TCP	SYN
2025-08-22T10:18:36.884481348Z	192.168.1.22	192.168.1.11	TCP	SYN
2025-08-22T10:18:36.898845225Z	192.168.1.22	192.168.1.11	TCP	SYN
2025-08-22T10:18:36.899184047Z	192.168.1.22	192.168.1.11	TCP	SYN
2025-08-22T10:18:36.902766989Z	192.168.1.22	192.168.1.11	TCP	SYN
2025-08-22T10:18:36.983103717Z	192.168.1.22	192.168.1.11	TCP	SYN
2025-08-22T10:18:36.983566918Z	192.168.1.22	192.168.1.11	TCP	SYN
2025-08-22T10:18:36.998407487Z	192.168.1.22	192.168.1.11	TCP	SYN
2025-08-22T10:18:36.999266963Z	192.168.1.22	192.168.1.11	TCP	SYN
2025-08-22T10:18:36.999710654Z	192.168.1.22	192.168.1.11	TCP	SYN
2025-08-22T10:18:37.000041698Z	192.168.1.22	192.168.1.11	TCP	SYN
2025-08-22T10:18:37.000353152Z	192.168.1.22	192.168.1.11	TCP	SYN
2025-08-22T10:18:37.000608082Z	192.168.1.22	192.168.1.11	TCP	SYN
2025-08-22T10:18:37.000860898Z	192.168.1.22	192.168.1.11	TCP	SYN
2025-08-22T10:18:37.000631464Z	192.168.1.22	192.168.1.11	TCP	SYN
2025-08-22T10:18:37.000955902Z	192.168.1.22	192.168.1.11	TCP	SYN
2025-08-22T10:18:37.001271926Z	192.168.1.22	192.168.1.11	TCP	SYN
2025-08-22T10:18:37.001503867Z	192.168.1.22	192.168.1.11	TCP	SYN
2025-08-22T10:18:37.001851070Z	192.168.1.22	192.168.1.11	TCP	SYN
2025-08-22T10:18:37.002151340Z	192.168.1.22	192.168.1.11	TCP	SYN

Figure 8: Captured Network Traffic

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.1.254	224.0.0.252	ICMP	240	Standard query 0x0000 PTR .services._dns-sd._udp.local, "QU" question PTR .companion-link._tcp.local, "QU" question PTR .rdln...
2	0.000000	192.168.1.254	224.0.0.252	ICMP	240	Standard query 0x0000 PTR .services._dns-sd._udp.local, "QU" question PTR .companion-link._tcp.local, "QU" question PTR .rdln...
3	0.000000	192.168.1.254	224.0.0.252	ICMP	240	Standard query 0x0000 PTR .services._dns-sd._udp.local, "QU" question PTR .companion-link._tcp.local, "QU" question PTR .rdln...
4	0.000000	192.168.1.254	224.0.0.252	ICMP	240	Standard query 0x0000 PTR .services._dns-sd._udp.local, "QU" question PTR .companion-link._tcp.local, "QU" question PTR .rdln...
5	0.000000	192.168.1.254	224.0.0.252	ICMP	240	Standard query 0x0000 PTR .services._dns-sd._udp.local, "QU" question PTR .companion-link._tcp.local, "QU" question PTR .rdln...
6	0.000000	192.168.1.254	224.0.0.252	ICMP	240	Standard query 0x0000 PTR .services._dns-sd._udp.local, "QU" question PTR .companion-link._tcp.local, "QU" question PTR .rdln...
7	0.000000	192.168.1.254	224.0.0.252	ICMP	240	Standard query 0x0000 PTR .services._dns-sd._udp.local, "QU" question PTR .companion-link._tcp.local, "QU" question PTR .rdln...
8	0.000000	192.168.1.254	224.0.0.252	ICMP	240	Standard query 0x0000 PTR .services._dns-sd._udp.local, "QU" question PTR .companion-link._tcp.local, "QU" question PTR .rdln...
9	0.000000	192.168.1.254	224.0.0.252	ICMP	240	Standard query 0x0000 PTR .services._dns-sd._udp.local, "QU" question PTR .companion-link._tcp.local, "QU" question PTR .rdln...
10	0.000000	192.168.1.254	224.0.0.252	ICMP	240	Standard query 0x0000 PTR .services._dns-sd._udp.local, "QU" question PTR .companion-link._tcp.local, "QU" question PTR .rdln...
11	0.000000	192.168.1.254	224.0.0.252	ICMP	240	Standard query 0x0000 PTR .services._dns-sd._udp.local, "QU" question PTR .companion-link._tcp.local, "QU" question PTR .rdln...
12	0.000000	192.168.1.254	224.0.0.252	ICMP	240	Standard query 0x0000 PTR .services._dns-sd._udp.local, "QU" question PTR .companion-link._tcp.local, "QU" question PTR .rdln...
13	0.000000	192.168.1.254	224.0.0.252	ICMP	240	Standard query 0x0000 PTR .services._dns-sd._udp.local, "QU" question PTR .companion-link._tcp.local, "QU" question PTR .rdln...
14	0.000000	192.168.1.254	224.0.0.252	ICMP	240	Standard query 0x0000 PTR .services._dns-sd._udp.local, "QU" question PTR .companion-link._tcp.local, "QU" question PTR .rdln...
15	0.000000	192.168.1.254	224.0.0.252	ICMP	240	Standard query 0x0000 PTR .services._dns-sd._udp.local, "QU" question PTR .companion-link._tcp.local, "QU" question PTR .rdln...
16	0.000000	192.168.1.254	224.0.0.252	ICMP	240	Standard query 0x0000 PTR .services._dns-sd._udp.local, "QU" question PTR .companion-link._tcp.local, "QU" question PTR .rdln...
17	0.000000	192.168.1.254	224.0.0.252	ICMP	240	Standard query 0x0000 PTR .services._dns-sd._udp.local, "QU" question PTR .companion-link._tcp.local, "QU" question PTR .rdln...
18	0.000000	192.168.1.254	224.0.0.252	ICMP	240	Standard query 0x0000 PTR .services._dns-sd._udp.local, "QU" question PTR .companion-link._tcp.local, "QU" question PTR .rdln...
19	0.000000	192.168.1.254	224.0.0.252	ICMP	240	Standard query 0x0000 PTR .services._dns-sd._udp.local, "QU" question PTR .companion-link._tcp.local, "QU" question PTR .rdln...
20	0.000000	192.168.1.254	224.0.0.252	ICMP	240	Standard query 0x0000 PTR .services._dns-sd._udp.local, "QU" question PTR .companion-link._tcp.local, "QU" question PTR .rdln...

Figure 9: Wireshark Validation of Captured Packets

The final validation stage compared packets captured by the Hybrid IDS with those observed in Wireshark. Packet timestamps, protocols, IP addresses, and TCP flags were analysed to verify that generated alerts accurately reflected the observed network activity.

Wireshark provides independent confirmation that packets processed by the IDS correspond to actual network communications. This comparison increases confidence in the accuracy of the packet capture module and confirms that the detection engine is operating correctly.

Successful correlation between IDS alerts and Wireshark packet captures demonstrates that the implemented solution performs reliable packet inspection, accurate attack detection, and effective security event logging.

Results and Discussion

The Hybrid Network Intrusion Detection System successfully achieved the objectives established at the beginning of the project. Throughout testing, the application continuously monitored live network traffic, analysed packet-level information,

detected multiple categories of malicious behaviour, and generated real-time security alerts without interrupting packet processing.

The system demonstrated the effectiveness of combining signature-based detection and anomaly-based detection within a single application. Signature-based techniques accurately identified known attack patterns such as TCP NULL scans, TCP XMAS scans, and suspicious port connections. At the same time, anomaly-based analysis successfully detected behavioural threats including port scanning activity, SYN flood attacks, and unusually high packet transmission rates. This hybrid detection strategy significantly improved the overall capability of the IDS by allowing it to recognise both predefined attack signatures and abnormal network behaviour.

All detected events were automatically recorded in structured CSV log files, providing detailed information such as timestamps, protocol types, source and destination IP addresses, port numbers, severity levels, and descriptive alert messages. These logs provide valuable forensic evidence and can be integrated into future monitoring or reporting systems.

Packet captures were independently verified using Wireshark, confirming that the IDS accurately interpreted observed network traffic. The correlation between generated alerts and packet-level analysis demonstrates the reliability of the detection engine and validates the implementation of the packet capture and analysis modules.

Overall, the project successfully demonstrates a practical implementation of a hybrid intrusion detection system capable of identifying common network attacks while providing useful security information for investigation and analysis.

Challenges Faced

Developing a real-time packet monitoring application presented several technical challenges throughout the implementation process. One of the primary difficulties involved configuring packet capture permissions and ensuring that the application could successfully access live network interfaces using the Pcap4J library. Proper installation and configuration of packet capture drivers were essential before traffic monitoring could begin.

Another challenge involved selecting appropriate thresholds for anomaly detection. If thresholds were configured too low, the IDS generated excessive false-positive alerts during normal network activity. Conversely, thresholds that were too high reduced the system's ability to detect suspicious behaviour promptly. Multiple rounds of testing and fine-tuning were required to achieve an effective balance between sensitivity and accuracy.

Validating generated alerts also required careful analysis. Every alert produced by the IDS was compared with captured packets in Wireshark to verify that detections corresponded to actual network events. This verification process improved confidence in the reliability of the detection logic and ensured that packet interpretation remained consistent throughout testing.

Performance optimisation represented another important consideration. Since packet capture operates continuously, the application needed to process incoming packets efficiently without introducing excessive delays. Implementing a modular architecture helped maintain responsiveness while allowing the system to perform packet analysis, detection, and logging simultaneously.

Skills Demonstrated

This project demonstrates practical technical skills across software development, networking, and cybersecurity. Throughout the implementation and testing process, knowledge from multiple technical domains was applied to develop a functional intrusion detection solution capable of monitoring live network traffic and identifying malicious activity.

Cybersecurity Skills

- Network Intrusion Detection
- Threat Detection
- Security Monitoring
- Packet Analysis
- Attack Simulation
- Incident Investigation
- Network Traffic Analysis

Programming Skills

- Java Development
- Object-Oriented Programming
- Exception Handling
- File Handling
- Modular Software Design
- CSV Data Processing

Networking Skills

- TCP/IP Protocol Analysis

- Packet Capture
- Network Interface Configuration
- Port Analysis
- TCP Flags
- Traffic Monitoring

Security Tools

- Pcap4J
- Wireshark
- Nmap
- hping3
- CSV Logging
- Command-Line Security Tools

Professional Skills

- Technical Documentation
- Problem Solving
- Analytical Thinking
- Troubleshooting
- Testing & Validation
- Cybersecurity Reporting

Future Improvements

Although the Hybrid Network Intrusion Detection System successfully demonstrates real-time packet monitoring and attack detection, several enhancements could further improve its capabilities and extend its use within enterprise environments.

One significant improvement would involve integrating Machine Learning algorithms capable of automatically learning normal network behaviour and detecting unknown attack patterns more accurately. Such an approach could reduce false positives while improving the identification of emerging threats.

Future versions could also integrate with Security Information and Event Management (SIEM) platforms such as Wazuh or Splunk. This integration would allow generated alerts to be centralised alongside logs from additional security devices, providing improved visibility across enterprise environments.

A graphical dashboard could also be developed to visualise network statistics, attack trends, protocol distribution, and alert history in real time. Such a dashboard would improve usability and provide security analysts with more intuitive monitoring capabilities.

Additional improvements may include:

- Email alert notifications
- Support for IPv6 traffic
- Detection of additional attack signatures
- Database storage for alert history
- REST API integration
- Cloud deployment
- Multi-threading optimisation
- Performance benchmarking
- Threat intelligence integration
- Automated report generation

These enhancements would significantly expand the practical applicability of the IDS while providing additional opportunities for future research and development.

Conclusion

This project successfully demonstrates the design, implementation, and evaluation of a Hybrid Network Intrusion Detection System (IDS) capable of monitoring live network traffic and identifying malicious activity through both signature-based and anomaly-based detection techniques.

By combining packet capture, behavioural analysis, predefined attack signatures, and structured alert logging, the developed solution provides an effective demonstration of modern intrusion detection principles. Controlled testing using industry-standard tools including Nmap, hping3, and Wireshark confirmed that the system successfully identified reconnaissance activities, suspicious network behaviour, SYN flood attacks, and abnormal packet rates while maintaining continuous traffic monitoring.

Beyond the technical implementation, this project highlights practical experience in Java development, packet analysis, network security, attack simulation, testing, troubleshooting, and technical documentation. The modular architecture also provides a strong foundation for future enhancements including machine learning integration, SIEM connectivity, dashboard visualisation, and cloud-based monitoring capabilities.

Overall, this project demonstrates the practical application of cybersecurity concepts to solve a real-world security problem. It reflects the ability to design, implement, validate, and document a complete intrusion detection solution suitable for professional portfolios, technical interviews, and GitHub publication.